PROJECT DECK & WHITEPAPER

AttestOps

SETTLED — VERIFIED

Cross-chain SLA settlement for DePIN — where device promises become on-chain obligations, and every claim is an Attestcoin proof.

TWO CONTRACTS · ONE THIN WORKER · ONE SCREEN · ONE STORY

COMPETITION	SECTOR	PROTOCOLS
BUIDL CTC 2026 Fall	DePIN	Creditcoin × Attestcoin

CHAINS	STATUS	SUBMISSION
Sepolia (source) · CC3 (settle)	Live on testnets · settled FULL	Deck PDF + demo video

Console: attestops.vercel.app Repo: github.com/Temmygabriel/Attestops 2026-08-31

01 · THE IDEA

An uptime promise that *settles itself*.

AttestOps lets a DePIN operator make a verifiable uptime commitment on one chain and have it proven and settled on another. Service facts — "my device held 98% uptime this window" — are emitted on **Sepolia**. A thin worker waits for **Attestcoin** to attest the source block, builds a single batch proof, and submits it to a settlement contract on **Creditcoin**. The contract verifies the proof itself against Creditcoin's **Block Prover precompile**, decodes the proven transactions, and applies deterministic SLA checks — all on-chain, with no trusted third party.

The problem

- **Claims are unverifiable.** Uptime numbers come from the operator's own server or a middleman's database — they cannot be audited.
- **Cross-chain reality is ignored.** A device serves on one chain while its economic agreement lives on another, so enforcement never reaches the money.
- **Settlement is non-deterministic.** No one can agree on what "good enough" means, or who gets paid what — enforcement is a negotiation, not a rule.

The result: **performance commitments are not assets.** They can't be escrowed, audited, or settled on their own terms.

02 · HOW IT WORKS

A self-settling on-chain obligation.

1. A device commits to a target uptime across **N service windows**, escrowing **collateral** + **reward**.
2. Each window's uptime is recorded as a `ServiceWindowClosed` event on the **source chain**.
3. **Attestcoin** cryptographically attests the source block.
4. The worker submits **one batch proof** covering all windows to the settlement contract.
5. The contract **verifies the proof itself** (precompile), **decodes the proven transactions**, and runs the SLA checks — atomically.
6. Settlement is deterministic: full pass pays full reward + collateral; partial pass pays in proportion; attacks are rejected wholesale.

No oracle. No trusted relayer. The proof *is* the data — facts only enter the contract from transactions the Block Prover precompile cryptographically proves were included on the source chain.

03 · PROTOCOL DEPTH

How the proof chain uses Attestcoin — end to end.

Depth of Attestcoin utilization is the design's core. Four distinct capabilities are used, in sequence:

CAPABILITY	WHERE	WHAT IT DOES
Attestation	PrecompileChainInfoProvider	Reads the latest attested height + hash from the `getLatestAttestedHeightAndHash` precompile — the source chain's finalized state.
Wait	waitUntilHeightAttested	Blocks until the target block is attested (resilient retry variant), so proofs are only built against finalized facts.
Batch proof	ProofBuilder.getBatchProof	Collapses **N transaction facts into one proof** with a shared continuity proof — the key to cheap cross-chain verification.
On-chain verification	Block Prover precompile 0x0FD2	The settlement contract itself calls `verify(...)` via staticcall. Facts only enter the contract from cryptographically proven transactions.

Why this is deep, not decorative

The contract does not receive "uptime numbers." It receives **raw proven transaction bytes** and decodes the `ServiceWindowClosed` event itself. Because the receipt — including `status == 1` (tx succeeded) and its logs — rides inside the proven bytes, a failed transaction, a forged log, or a reordered window cannot survive verification. The trust boundary is **cryptographic, not procedural**.

Source registry on Sepolia, settlement on Creditcoin.

ServiceRegistry.sol · source chain

Registers devices, creates windows, closes them with the operator's self-reported uptime.

```
registerDevice(deviceId)
createServiceWindow(deviceId, uptimeBps,
reward) → windowId
closeServiceWindow(deviceId, windowId)
```

Emits `ServiceWindowClosed(bytes32 indexed deviceId, uint256 indexed windowId, uint256 uptimeBps, uint256 rewardAmount)`

SLASettlement.sol · settlement chain

The economic agreement: escrows collateral + reward, verifies batch proofs, applies checks, settles.

```
createSLA(slaId, deviceId, emitter,
reqWindows, minUptimeBps, collateral,
reward) payable
submitProvenBatch(slaId,
chainKey, heights[], txBytes[],
proofs[], continuity)
settle(slaId) · slash(slaId) ·
owner governance
```

Deterministic, reproducible identity

```
deviceId = keccak256(deviceName)
slaId    = keccak256(deviceId ‖ keccak256("attestops-sla-v1"))
```

One SLA per commitment period

A source window transaction can be proven for **exactly one** SLA (replay protection). Starting a new commitment means registering a new device — enforced by design, not by convention.

05 · DETERMINISTIC LOGIC

Five checks per fact, atomically across the batch.

On every `submitProvenBatch`, the contract verifies the proof first, then applies:

#	CHECK	WHAT IT STOPS
1	**Proof gate**	Batch must verify against the precompile, or the whole submission reverts (`ProofVerificationFailed`).
2	**Source identity**	The proven event's emitter must be the registered source emitter.
3	**Device binding**	The proven event must belong to this SLA's device.
4	**Strict ordering**	Windows must arrive in sequence; a gap or repeat is rejected.
5	**Replay**	Each proven transaction may be consumed once, ever (`ReplayDetected`).

Additional gates: correct chain key, length consistency, non-empty batches, no submission after settlement or completion, and reverted or forged transactions cannot slip through the decode.

Settlement economics

OUTCOME	CONDITION	PAYOUT
FULL	all required windows pass	full reward + full collateral returned
PARTIAL	some windows pass	reward and collateral scaled by `passed / required`
NONE	abandoned / never completed	owner `slash` can seize the escrow

The escrow (collateral + reward) is locked at `createSLA` and released on-chain at settlement — no multisig, no operator, no waitlist.

06 · SECURITY

The threat model: no party is trusted.

ATTACK	DEFENSE (EXACT REVERT)
Operator claims uptime it didn't achieve	Facts are proven receipts; `status == 1` and logs are inside the proof — can't be forged
Replaying a good proof on a second SLA	`consumedSourceTx` — one tx, one SLA, ever · `ReplayDetected`
Submitting an out-of-order history	strict sequential ordering · `OutOfOrder`
Emitting facts from the wrong contract	emitter whitelist + per-fact match · `WrongEmitter`
Binding another device's windows	per-fact device match · `WrongDevice`
Below-threshold windows hiding a miss	threshold check — recorded as failed, settles partial
A poisoned fact sneaking into a good batch	**batch atomicity** — one bad fact reverts the entire batch's state
Trusting a batch that doesn't verify	proof gate first, revert · `ProofVerificationFailed`

Enforced by a **63-test adversarial suite** — batch atomicity, replay, wrong emitter / device, ordering, below-threshold, incomplete history, reverted txs, forged txBytes.

07 · PROOF OF WORKING

Real on-chain record — live testnets, settled FULL.

Every hash is a real transaction you can open in an explorer. Both contracts deployed by the same wallet at the same CREATE address: `0xB6daA5aDeeB208EBFf91FB2636E86B9dc3aEbE45` (Sepolia ServiceRegistry · CC3 SLASettlement, chain key 1, chain id 102031).

NODE-010 — the canonical demo

`10 windows · 98% min · 100/40 CTC · ONE batch proof`

STEP	TRANSACTION	GAS
Create SLA (escrow 140 CTC)	`0x06e27d67…c4f80`	193,201
One batch proof → 10 WindowVerified	`0xecb9973f…6ff2fa`	673,145
Settle → FULL, 140 CTC returned	`0xbfcc3362…d80967`	115,052

Source facts: Sepolia blocks **11602226–11602245**, uptimes 9850–9950 (all ≥ 9800) · SLA `0x978ccb4f…` · device NODE-010 · contract state confirms `verified=10, passed=10, outcome=FULL`.

NODE-015

`2 windows · FULL`

Submit `0xd38f7eec…` (245,858 gas) → settle `0xd25fe8fe…`
SLA `0xe20e839b…`

NODE-014

`4 windows · FULL`

Submit `0x002e22c6…` (408,701 gas) → settle `0x2941017d…`
SLA `0x832b3756…`

Gas is metered on-chain per step — the batch proof of 10 windows costs roughly the same as the batch proof of 4, which is the point of batch verification.

08 · ONE SCREEN

The console: the whole mechanism, one page.

Deployed at attestops.vercel.app, rendered in the Notary Ledger design language — aged paper, ruled ledger, rubber-stamp verdicts — so the demo screen and this deck tell the same story.

- **Commitment** — device, target uptime, windows, escrow, reward.
- **Proof chain** — the four hops Source → Attest → Prove → Settle, each stamped VERIFIED.
- **Checkpoints** — one ruled ledger row per window: id, uptime bar, passed / failed pill.
- **Settlement** — the rotated FULL PASS (or PARTIAL) seal, payout math, escrow release.
- **Threat lab** — overlays the live state with what the contract rejects: replay, below-target, proof-fails — the exact revert reasons the test suite asserts.

```
Read-only and static: public RPCs only, zero secrets, embedded verified snapshot fallback so
the story renders even if live reads are unavailable.
```

Engineering

63 unit tests — three suites: ServiceRegistry, SLASettlement, SettlementValidation (adversarial). Foundry.

Replay-safe worker — computes `keccak256(txBytes)` replay keys from the proof and drops already-consumed transactions before paying gas.

Deployment, roadmap, and how to reach it.

Deployment

- `vercel.json` → static HTML, no build, no environment variables.
- Testnet wallet key exists only in local `.env` (gitignored) for the operator fixture tooling.
- The deployed surface carries **zero secrets**.

Roadmap

- Multi-chain sources (any attested chain) + per-SLA source selection.
- Programmatic rewards: token payouts, staking on escrow.
- Event-driven worker; multi-SLA batching.
- Partial-failure clawbacks and dispute windows.
- Mainnet readiness: audit, gas-optimized decode.

Links

Console · attestops.vercel.app

Repo · github.com/Temmygabriel/Attestops

Precompile · Block Prover 0x0FD2

Chains · Sepolia + CC3 (102031)

FULL PASS

VERIFIED

REJECTED

Two contracts · one thin worker · one screen · one story.